

Data Structures and Algorithms Using C++

Folder: <https://shorturl.at/psDW7>

LU2: Analyze Data Structures and algorithms

2.1 Algorithm Analysis

2.1.1 Introduction: What is an Algorithm?

=>An **Algorithm** is a finite set of unambiguous steps or instructions to solve a given problem.

=>An algorithm is a **set of instructions for solving a problem** or accomplishing a task.

=>It can also be defined as a process or **set of rules to be followed** in calculations or other problem-solving operations, especially by a computer. Knowledge of algorithms helps us to get desired results faster by applying the appropriate algorithm. We learn by experience. With experience, it becomes easy to solve new problems.

The properties of an algorithm are:

1. It takes **zero or more inputs**.
2. It should produce **one or more output**.
3. It should be **Deterministic**. It produces the same output if the same input is provided again.
4. It should be **Correct**. It should be correct and able to process all the given inputs and provide the correct output.
5. It should **Terminate** in a finite time.
6. It should be **Efficient**. The algorithm should be efficient in solving problems.

2.1.2 Algorithm representation:

- Natural Language
- Pseudocode
- Flowchart

2.1.3 Algorithm Complexity

The **complexity of an algorithm** is the amount of **Time** or **Space** required by the algorithm to process the input and produce the output. Algorithm designers strive to develop algorithms with the lowest possible time and space/memory complexities, since this makes them more efficient and scalable.

There are **two types of Complexity**:

1. First is **Time-Complexity**, how much time is required by an algorithm to produce output for an input of size 'n'.

Time-Complexity is represented by function $T(n)$ - time required versus the input size n .

2. Second is **Space-Complexity**, how much RAM or memory that an algorithm is going to consume to produce output for an input of size 'n'.

Space-Complexity is represented by function $S(n)$ - memory used versus the input size n .

An algorithm is analyzed using Time Complexity and Space Complexity. Writing an efficient algorithm help to consume the minimum amount of time for processing the logic.

Experimental Analysis

Write code for every solution, record the time and plot the time on the graph. Compile and run the following program for both Merge Sort and selection sort for 10 million elements:

<https://shorturl.at/ILjWN>

Selection sort: 10.000.000 elements

Merge Sort for 10.000.000 elements

```
Merge sort n = 10 time = 3
Merge sort n = 100 time = 26
Merge sort n = 1000 time = 265
Merge sort n = 10000 time = 3381
Merge sort n = 100000 time = 21144
Merge sort n = 1000000 time = 106378
Merge sort n = 10000000 time = 1140454
```

```
Selection sort n = 10 time = 3
Selection sort n = 100 time = 57
Selection sort n = 1000 time = 4751
Selection sort n = 10000 time = 123042
Selection sort n = 100000 time = 9104846
Selection sort n = 1000000 time = 1087351129
```



The selection sort to finish sorting 100 Million elements will take 8-10 hours which is very difficult to wait or handle. Experimental analysis is difficult to use.

Theoretical analysis of an Algorithm

They are 3 solutions. Without writing the code, you should be able to tell which is the best and the time complexity of every solution as well as the time function: **Asymptotic Analysis or Asymptotic notations**

Asymptotic Analysis or Asymptotic notations

Calculating the running time of any algorithm in mathematical units of computation is known as Asymptotic Analysis. The efficiency of algorithms is calculated using asymptotic analysis, independent of the given data set or programming language.

In most cases, we are interested in the order of growth of the algorithm instead of the exact time required for running an algorithm. This time is also known as Asymptotic Running Time.

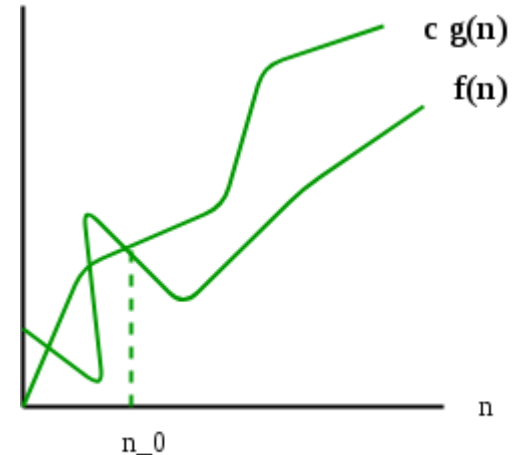
The performance of an algorithm can change with a change in the input size. That is where **Asymptotic Notations** like **Big O Notation** comes into play. Asymptotic Notations can describe an algorithm's run time when the input tends toward a specific or limiting value. Asymptotic analysis helps to analyze the algorithm performance change in the order of input size.

Big O Notation

Formal definition: " $f(n)$ is big-O of $g(n)$ " or $f(n) = O(g(n))$, if there are two constants c and n_1 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_1$; in other words, $c \cdot g(n)$ is an upper bound for $f(n)$ for all $n \geq n_0$. The function $f(n)$ growth is slower than $c \cdot g(n)$. For a sufficiently large value of input n , the $(c \cdot g(n))$ will always be greater than $f(n)$.

Big-O notation in Data Structure represents the upper bound of the running time of an algorithm or the Worst case complexity.

Big O Notation is a tool used to describe the time complexity of algorithms. It calculates the time taken to run an algorithm as the input grows.



$$f(n) = O(g(n))$$

**Mathematical
Representation of Big-
O**

Big O Notation(next)

The formal definition is useful when you need to perform a math proof. For example, the time complexity for selection sort can be defined by the function $f(n) = n^2/2 - n/2$ as we have discussed in the previous section.

If we allow our function $g(n)$ to be n^2 , we can find a constant $c = 1$, and a $N_0 = 0$, and so long as $N > N_0$, N^2 will always be greater than $N^2/2 - N/2$. We can easily prove this by subtracting $N^2/2$ from both functions, then we can easily see $N^2/2 > -N/2$ to be true when $N > 0$. Therefore, we can come up with the conclusion that $f(n) = O(n^2)$, in the other selection sort is “big O squared”.

You might have noticed a little trick here. That is, if you make $g(n)$ grow super fast, way faster than anything, $O(g(n))$ will always be great enough. For example, for any polynomial function, you can always be right by saying that they are $O(2^n)$ because 2^n will eventually outgrow any polynomials.

Mathematically, you are right, but generally when we talk about Big O, we want to know the **tight bound** of the function.

Big O Notation for Time Complexity: Example

Consider the case of Bubble Sort. If the array of **N** is not sorted, To swap two elements it take **k** amount of time, and this must happen **N-1** time; the first element will be swapped with **N-1** and take **k(N-1)** time, the second must be swapped with **N-2** elements which means **k(N-2)**, the last element will be swapped with **N-N=0=K** amount of time.

1	2	3	4	5	...	..	N
---	---	---	---	---	-----	----	---

$$T(N)=k(N-1)+k(N-2)+k(N-3)+.....+K$$

$$T(N)=K(1+2+3+4+.....+n-1) \text{ // As per PMI, the sum of } n \text{ first numbers is } \sum n=n(n+1)/2$$

$$T(N)=K(N-1)*N/2$$

$$T(N)=KN^2/2-K(N)$$

The constant $K/2$ will be ignored. And keep N^2 ,

This means that the time complexity for Bubble Sort is $O(N^2)$.

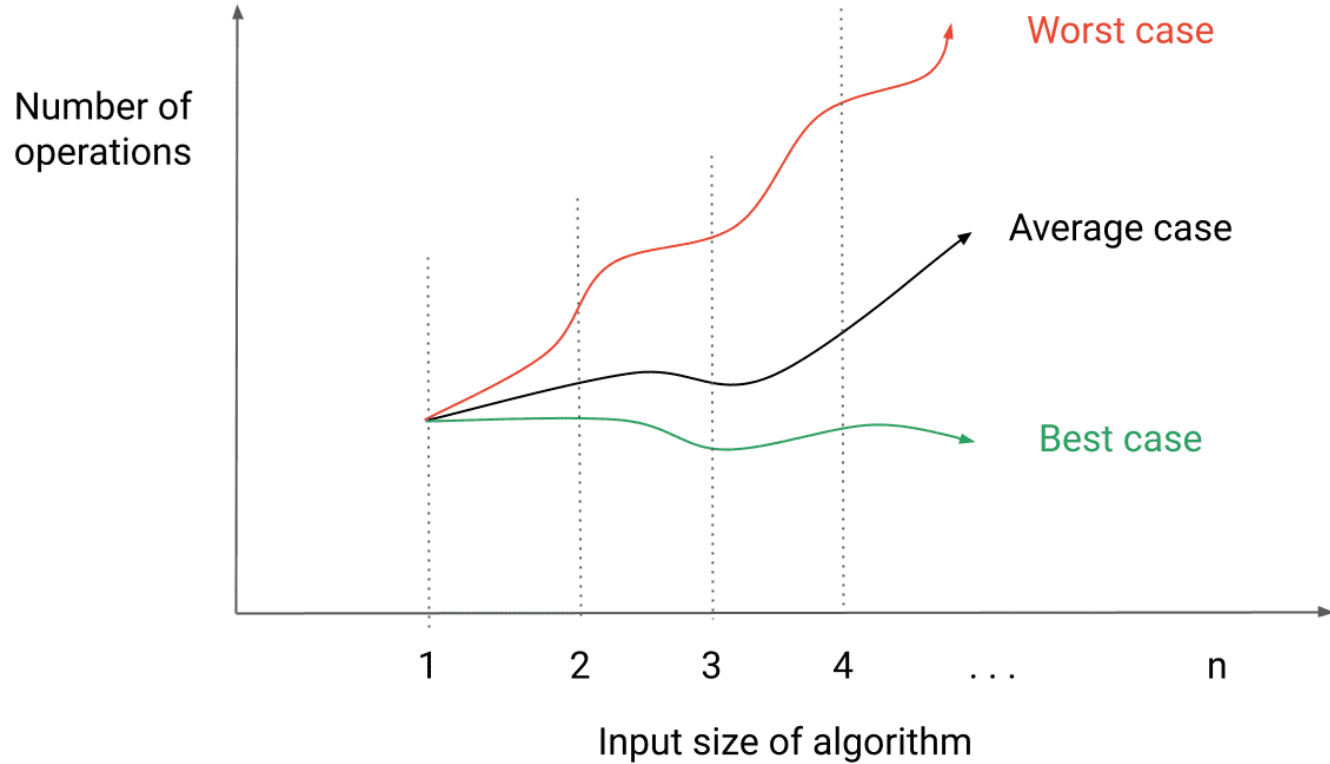
Complexity analysis of algorithms

The complexity of an algorithm are analysed in three categories:

- **Worst-Case Complexity:** The worst-case complexity represents the maximum number of steps required to execute an algorithm. It provides us with the upper bound of an algorithm. Usually, we use this complexity to judge algorithm performance.
- **Best-Case Complexity:** The best-case complexity represents the minimum number of steps required to execute an algorithm. It provides us with the lower bound of an algorithm.
- **Average-Case Complexity:** The average-case complexity represents the average number of steps required to execute an algorithm. We take the average of the steps executed in all the cases to calculate average-case complexity.

Note: Worst-case complexity is used to find the guarantee in how much time some particular algorithm will finish. This is the **most important time complexity**. If the type of complexity is not mentioned, then always consider Worst-Case time complexity.

Graphical representation of the Three categories



Growth Functions

Let us look at these growth rates of various functions. The size of input is n

Constant Time, $O(1)$

An algorithm is said to run in constant time if the output is produced in constant time, regardless of the input size.

Examples: Accessing an n^{th} element of an Array

Linear Time, $O(n)$

An algorithm is said to run in linear time if the execution time of the algorithm is directly proportional to the input size.

Examples:

1. Array operations like search element, find min, find max etc.
2. Linked list operations like traversal, find min, find max etc.

Note: If we need to traverse all the nodes of a data structure for some task, then complexity can't be less than $O(n)$

Growth Functions

Logarithmic Time, $O(\log(n))$

An algorithm is said to run in logarithmic time if the execution time of the algorithm is proportional to the logarithm of the input size. In each step of an algorithm, a significant portion (e.g. half portion) of the input is pruned/rejected out without traversing it.

An example is the **Binary search algorithm**. We will read about this algorithm later.

$n.\log(n)$ Time, $O(n.\log(n))$

An algorithm is said to run in $n \cdot \log(n)$ time if the execution time of an algorithm is proportional to the product of input size and logarithm of the input size. In these algorithms, each time the input is divided into half (or some proportion) and each portion is processed independently.

Examples: Merge-Sort, Quick-Sort (average case), Heap-Sort etc.

Growth Functions

Quadratic Time, $O(n^2)$

An algorithm is said to run in quadratic time if the execution time of an algorithm is proportional to the square of the input size. In these algorithms, each element is compared with all the other elements.

Examples: Bubble-Sort, Selection-Sort, Insertion-Sort

Exponential Time $O(2^n)$

In these algorithms, all possible subsets of elements of input data are generated. Its common example is the power set.

Factorial Time $O(n!)$

In these algorithms, all possible permutations of the elements of input data are generated. Finding permutations is a common example of factorial time.

A list of commonly occurring algorithm Time Complexity in increasing order

Commonly encountered time complexities in increasing order

Name	Running Time(T_n)
Constant	$O(1)$
Logarithmic	$O(\log(n))$
Linear	$O(n)$
$n \cdot \log(n)$	$O(n \cdot \log(n))$
Quadratic	$O(n^2)$
Polynomial	$O(n^c)$ c is a constant and $c > 1$
Exponential	$O(c^m)$ c is a constant and $c > 1$
Factorial	$O(n!)$
N power N	$O(n^n)$

With Big O Notation:

1. We ignore constants
2. Focus on the highest power

Examples

1. $7n^2 + 5n + 8 \Rightarrow O(n^2)$
Because $n^2 \gg n$
2. $n^3 + n^2 \Rightarrow O(n^3)$
3. $5n^n + n^3 \Rightarrow O(n^n)$
4. $5n^2 + n \cdot \log(n) \Rightarrow O(n^2)$
5. $n + n \log(n) \Rightarrow O(n \log(n))$

Functions Growth Rate(Approximation)

The time taken by certain algorithms to run varies dramatically with the size of the input. Some algorithms take minutes or even seconds to run on huge input, whereas others may take days to complete their execution. To understand how the rate of growth changes with the size of the input in different functions, the following table presents the approximate number of steps required to run an algorithm

N	Function Growth Rate (Approximate)						
	O(1)	O(log(n))	O(n)	O(n.log(n))	O(n ²)	O(n ³)	O(2 ⁿ)
10	1	3	10	30	10 ²	10 ³	10 ³
10 ²	1	6	10 ²	6x10 ²	10 ⁴	10 ⁶	10 ³⁰
10 ³	1	9	10 ³	9x10 ³	10 ⁶	10 ⁹	10 ³⁰⁰
10 ⁴	1	13	10 ⁴	13x10 ⁴	10 ⁸	10 ¹²	10 ³⁰⁰⁰
10 ⁵	1	16	10 ⁵	16x10 ⁵	10 ¹⁰	10 ¹⁵	10 ³⁰⁰⁰⁰
10 ⁶	1	19	10 ⁶	19x10 ⁶	10 ¹²	10 ¹⁸	10 ³⁰⁰⁰⁰⁰

Deriving/Hints an Algorithm's Runtime Function

Constants

If any line of code is a statement with basic operations, e.g., comparisons, assignments, or reading a variable, they take constant time each. Thus, the time complexity of each statement is $O(1)$.

Loops

In loop, a repetition of a particular code for n times, where n is the size of the loop. Every statement inside the loop has a runtime of $O(1)$. The running time of a loop is a product of the running time of the statement inside a loop and the number of iterations in the loop. Time Complexity is $O(n)$

Nested Loops

The running time of nested loops is a product of the running time of the statements inside the loop multiplied by a product of the size of all the loops. Time Complexity is $O(n^c)$. Where c is the number of loops. For two loops, it will be $O(n^2)$

Consecutive Statements

In this case, we add the running time of all the consecutive lines of code.

If-Else Statement

In this case, either "if" will run or "else" will run. So, the block with larger runtime will be considered.

Deriving an Algorithm's Runtime Function

Logarithmic Statement

In this case, each iteration will cut the input size into b pieces and consider one of the pieces for the next iteration. Time complexity in this situation will be $O(\log(n))$.

Time Complexity Examples

Example 2.1: Single loop

```
int fun1(int n) {  
    int m = 0;  
    for (int i = 0; i < n; i++) {  
        m += 1;  
    }  
    return m;  
}  
  
// Testing Code  
int main() {  
    std::cout << "N = 100, Number of instructions O(n) :: " << fun1(100);  
    return 0;  
}
```

Output:

N = 100, Number of instructions in O(n): :100

Time Complexity: O(n), single loop takes linear time

Example 2.2: Nested loops

```
#include <iostream>  
using namespace std;  
int fun6 (int n) {  
    int m = 0;  
    for(int i=0;i<n;i++){  
        for(int j=0;j<n;j++){  
            m += 1;  
        }  
    }  
    return m;  
}  
  
// Testing Code  
int main() {  
    std::cout << "N = 100, Number of instructions  
O(n^2)::fun2(100);  
    return 0;  
}
```

Output:

N = 100, Number of instructions in O(n^2)::10000

Time Complexity: O(n²), two nested loop, takes quadratic time. Both the loop is executed in number of times, so the internal statement executed n² number of times

Time Complexity Examples

Example 2.3: Nested Loop

Example 1.3: Nested loops

```
int fun3(int n) {  
}  
int i, j, k, m = 0;  
for (i = 0; i < n; i++) {  
    for (j = 0; j < n; j++) {  
    }  
    for (k = 0; k < n; k++) {  
        m += 1;  
    }  
}  
return m;  
}  
// Testing Code  
int main() {  
    std::cout << "N = 100, Number of instructions O(n^3) :: " <<  
    fun3(180);  
    return 0;  
}
```

Output:
N = 100, Number of instructions in $O(n^3)$:: 1000000
<< fun2(100);

Time Complexity: $O(n^3)$, All the three nested loops run for n number of iterations. So the statement inside the innermost loop will run for n^3 number of times.

Example 2.4: Arithmetic Progression

```
#include <iostream>  
using namespace std;  
int fun6 (int n) {  
    int i,j, m = 0;  
    for(i=0;i<n;i++){  
        for(j=0;j<i;j++){  
            m += 1;  
        }  
    }  
    return m;  
}  
// Testing Code  
int main() {  
    std::cout << "N = 100, Number of instructions O(n^2) :: << fun4 (100);  
    return 0;  
}
```

Output:

N = 100, Number of instructions in $O(n^2)$::4950

Time Complexity: Statement inside inner loop executes for 1 time in first iteration then 2 times then 3 times and so on for n iterations. Total number of times the inner statement is executed = $1 + 2 + 3 + \dots + n$.

This series is an arithmetic progression, which sums to $n(n+1)/2$. So the final time complexity is $O(n(n+1)/2)$ ignoring the constant factors, time complexity will be $O(n^2)$.

Time Complexity Examples

Example 2.5 Geometric Progression

```
int fun5(int n) {
    int i, j, m = 0;
    for (i = 1; i <= n; i *= 2) {
        for (j = 0; j <= i; j++) {
            m += 1;
        }
    }
    return m;
}
```

0122

}

// Testing Code

```
int main() {
    std::cout << "N = 100, Number of instructions::"<< fun5
(100); 0(n)
    return 0;
}
```

Output:

N = 100, Number of instructions in $O(n)$:: 134

Time Complexity: The inner loop will run for 1, 2, 4, 8,... n times in successive iteration of the outer loop.
 $T(n) = O(1 + 2 + 4 + \dots + n/2 + n) = O(n)$

Example 2.6: Double the iteration variable

```
#include <iostream>
using namespace std;
int fun6 (int n) {
    int i = 1, m = 0;
    while (i < n) {
        m += 1;
        i = i * 2;
    }
    return m;
}
// Testing Code
int main() {
    printf("N = 100, Number of instructions  $O(\log(n))$  :: %d \n", fun6
(100));
    return 0;
}
```

Output:

N = 100, Number of inst in $O(\log(n))$:: 7

In each iteration, i value is doubled. So the value of i after k iterations will be 2^k .

$2^k = n$... Will be the condition at the time of exit.

$\log(2^k) = \log(n)$...Taking log both sides.

$k = \log(n)$

Time Complexity: $O(\log(n))$

Time Complexity Examples

Example 2.7: Consecutive Statements

```
int fun7(int n) {
int i, j, k, m = 0;
for (i = 0; i < n; i++) {

for (j = 0; j < n; j++) {
m += 1;
}
}
for(i = 0; i < n; i++) {
for (k = 0; k < n; k++) {
m += 1;
}
}
return m;
}
// Testing Code
int main() {
std::cout << "N = 100, Number of instructions O(n^2):: "<<
fun7(100);
return 0;
}
```

Output:
N = 100, Number of instructions in $O(n^2)$::20000

These two groups of loops are consecutive, so their

complexity will add up to form the final complexity of the program.

Time Complexity: $O(n^2) + O(n^2) = O(n^2)$

Problem: Given a value N find $N!$. Where $N! = N * (N-1) * \dots * 2 * 1$. Use recursion to solve the problem.

Example 2. 8: Factorial Calculation.

```
int factorial (int i) {
/* Base Case or Termination Condition */
if (i <= 1)
}
return 1;
/* Body, Recursive Expansion */
return i *factorial (i - 1);
}
int main() {
std::cout << "Factorial: " << factorial (5) << std::endl;
return 0;
}
```

Output:

Factorial:120

Analysis: We calculate factorial(i) as i*factorial (i-1) recursively.

Function $F(n)$ calls $F(n-1)$

$$T(n) = T(n-1) + 1$$

$$T(n-1) = T(n-2) + 1$$

$$T(n-2) = T(n-3) + 1$$

$$T(n) = T(n-1) + 1 = (T(n-2) + 1) + 1 = T(n-2) + 2 = (T(n-3) + 1) + 2 =$$

$$T(n-3) + 3 \text{ Similarly, for } k^{\text{th}} \text{ term } T(n) = T(n-k) + k$$

for base case $(n-k) = 1$ or $n-1=k$

$$T(n) = T(1) + n-1 = n$$

Time Complexity : $O(n)$

Time Complexity Examples

Fibonacci number

Problem: Given N, find the Nth number in the Fibonacci series.

Solution: Fibonacci numbers are calculated by adding the sum of the previous two numbers.

Example 1.10:

```
int fibonacci(int n) {  
    if (n < 2) {  
        return n;  
    }  
    return fibonacci(n-1) + fibonacci(n - 2);  
}  
  
int main() {  
}  
std::cout << fibonacci (10) << std::endl;  
return 0;  
}
```

Output:

55

Analysis: Recurrence Relation: $T(n) = 1 + T(n-1) + T(n-2)$

$T(n) = 1 + 2T(n-1)$ // Approximately

$T(n) = 1 + 2T(n-1)$

$T(n-1) = 1 + 2T(n-2)$

$T(n-2) = 1 + 2T(n-3)$

$T(n) = 1 + 2(1+2T(n-2)) = 1 + 2 + 4T(n-2) = 1 + 2 + 4(1 + 2(T(n-3))) = 1 + 2 + 2^2 + 2^3T(n-3)$

$= 1 + 2 + 2^2 + \dots + 2^nT(0) = 1(2^{n+1}-1)/2-1 = 2^{n+1} - 1$

Time complexity is $O(2^n)$, ignoring the constants.

Time Complexity Examples

Problem: n-digit cast iron padlock

```
FOR i←0 To 10
  FOR j←0 To 10
    .....
    .....
    nth loop
    IF code = combination
      BREAK
    ENDFOR (nth loop)
  ENDFOR
  .....
  .....
ENDFOR
```

Time Complexity Examples

In this algorithm if n is 3, we'll have 3 loops going from 0 to 9, therefore $10 \times 10 \times 10$ iterations, or **10^3 iterations**, for $n=4$, we'll have 4 loops going from 0 to 9, therefore $10 \times 10 \times 10 \times 10$ iterations, or **10^4 iterations**.

With Big O notation, we will say that the time complexities of this algorithm $O(10^3)$ and $O(10^4)$ respectively.

We therefore see that **our safe opening algorithm depends on the number of digits in the code**.

If we note n the number of digits of our padlock, the calculation time complexity is a function of 10^n ; thus, using the notion **big O**, we can note that the time complexity of our algorithm is **$O(10^n)$** .

We then say that **the complexity is exponential and is noted $O(\log(n))$** .

Other Asymptotic Notations for Time/Space Complexity

Omega Notation – Ω

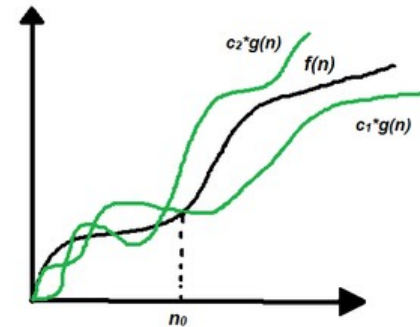
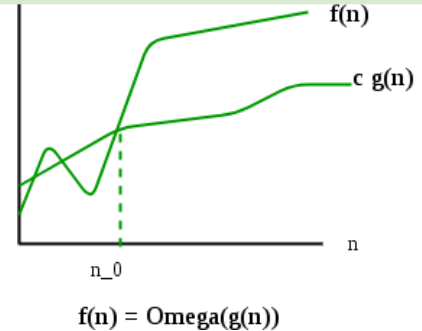
it gives the **best-case scenario of an algorithm's complexity**, opposite to big-O notation. We can say that: “the amount of Time/space this algorithm takes will grow no more slowly than this *fix*), but it could grow more quickly.”

Theta Notation – θ

Theta notation encloses the function from above and below. Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the average-case complexity of an algorithm

Theta (Average Case) You add the running times for each possible input combination and take the average in the average case.

Mathematical Representation of **Omega** and **Theta**



2.1.4 Space Complexity

Space complexity refers to the total amount of memory space used by an algorithm/program, including the space of input values for execution. Calculate the space occupied by variables in an algorithm/program to determine space complexity. The best algorithm/program should have a low level of space complexity. The less space required, the faster it executes.

Notation

There are different notations we can use to express space complexity measurements. The most widely used is

Big-O Notation describes an asymptotic upper bound. It represents the algorithm's scalability and performance.

It gives the worst-case scenario of an algorithm's growth rate. We can say that: "the amount of space this algorithm takes will grow no more quickly than this $f(x)$, but it could grow more slowly."

Let's see a few examples of expressing space complexity using big-O notation, starting from slowest space growth (best) to fastest (worst):

$O(1)$ – constant complexity – takes the same amount of space regardless of the input size

$O(\log n)$ – logarithmic complexity – takes space proportional to the *log* of the input size

$O(n)$ – linear complexity – takes space directly proportional to the input size

$O(n \log n)$ – log-linear/quasilinear complexity – also called "linearithmic", its space complexity grows proportionally to the input size and a logarithmic factor

$O(n^2)$ – square/polynomial complexity – space complexity grows proportionally to the square of the input size

Analyzing Space Complexity of Algorithms

The ability to calculate space complexity is essential in considering an algorithm's efficiency. In this section, we'll analyze the space complexity of a few programs of differing difficulty. We'll measure it using big-O notation.

Above all, it's necessary to mention that **space complexity depends on a variety of things such as the programming language, the compiler, or even the machine running the algorithm.**

We'll use C++ for our examples, although, each of them could be easily applied to any technology. To apply them to other programming languages, we can simply replace the C++ data types with language-specific ones. Then in our calculations, we'd use the sizes that correspond to the language-specific data types.

Example1: Space Complexity

Consider the simple program that sums two integers (numbers without a fractional part):

```
int addition(int a, int b) {  
    return a + b;  
}
```

Copy

In this particular method, three variables are used and allocated in memory:

- the first *int* argument, *a*
- the second *int* argument, *b*
- and the returned sum result which is also an *int* like *a* and *b*

A single integer variable occupies four bytes of memory. In this example, we have three integer variables. Therefore, this algorithm always takes 12 bytes of memory to complete (3*4 bytes).

We can clearly see that the space complexity is constant, so, it can be expressed in big-O notation as $O(1)$.

Example2: Space Complexity

Determine the space complexity of a program that sums all integer elements in an array.

Let's list all variables present in the above code:

- *array* – the function's only argument – the space taken by the array is equal $4n$ bytes where n is the length of the array
- *size* – a 4-byte integer
- *sum* – a 4-byte integer
- *iterator* – a 4-byte integer

The total space needed for this algorithm to complete is $4n + 4 + 4 + 4$ (bytes). The highest order of n in this equation is just n . Thus, the space complexity of that operation is $O(n)$.

```
#include <iostream>
using namespace std;
int sumArray(int a[], int size) {
    int sum = 0;
    for (int iterator = 0; iterator < size;
iterator++)
    {
        sum += a[iterator];
    }
    return sum;
}
int main() {
    int arr[5]={10,20,30,40,50};
    int size = sizeof(a)/sizeof(a[0]);
    cout << "The sum is
"<<sumArray(arr,size)<<endl;
    return 0;
}
```

Conclusion

1. Time Complexity:

Time complexity is a measure of the amount of time an algorithm takes with respect to the size of its input. It helps in understanding how the running time of an algorithm increases as the size of the input increases. The common notations used for denoting time complexity are:

Conclusion(Cont)

- O (Big O) Notation: It gives the upper bound of the time an algorithm takes to run, in the worst-case scenario.
- Ω (Big Omega) Notation: It gives the lower bound of the time an algorithm takes to run, in the best-case scenario.
- Θ (Theta) Notation: It provides an average case time complexity.

Conclusion(Cont)

Common time complexities include:

- Constant Time ($O(1)$): The algorithm's running time is constant and does not depend on the size of the input.
- Linear Time ($O(n)$): The running time of the algorithm increases linearly with the size of the input.
- Logarithmic Time ($O(\log n)$): The running time of the algorithm increases logarithmically with the size of the input.
- Quadratic Time ($O(n^2)$): The running time of the algorithm increases quadratically with the size of the input.
- Exponential Time ($O(2^n)$): The running time of the algorithm increases exponentially with the size of the input.

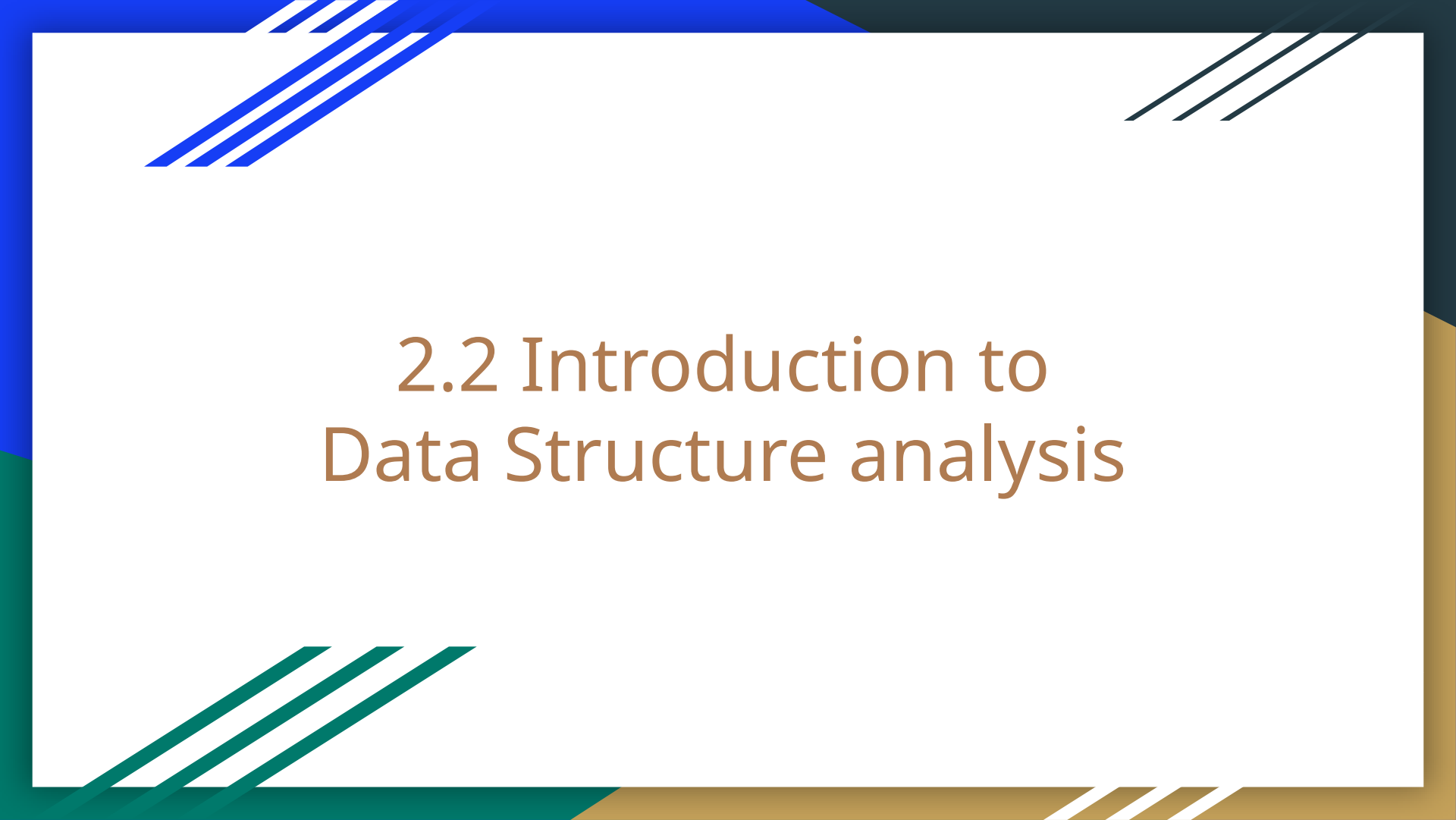
Conclusion(Cont)

2. **Space Complexity:**

Space complexity refers to the amount of memory space that an algorithm requires to solve a problem. It's a measure of the maximum amount of memory space used by the algorithm for a given problem size. Similar to time complexity, space complexity can also be denoted using Big O notation.

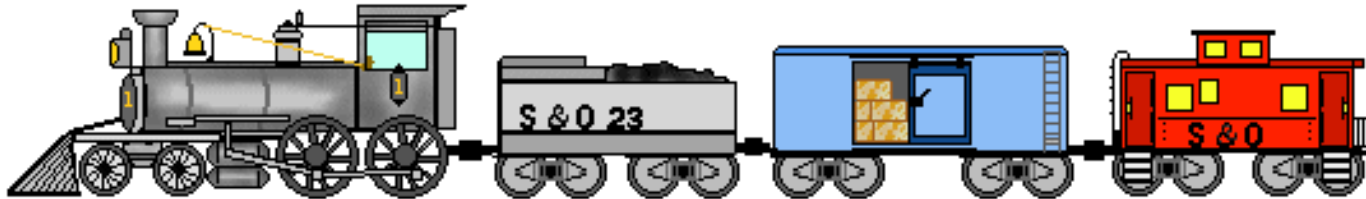
Conclusion(Cont)

It's important to analyze both time and space complexities while evaluating algorithms, as a trade-off between the two may often exist. Choosing the right algorithm involves considering both time and space complexities, along with the specific requirements and constraints of the problem at hand.



2.2 Introduction to Data Structure analysis

Observe the figures and describe what you see;



2.2.1 Introduction to Data Structures

Data Structure is a logical or mathematical model for collecting and organizing data in such a way that we can perform operations on these data in an effective way. Data Structures is about rendering data elements in terms of some relationship, for better organization and Storage.

Example: Library is composed of elements (books) to access a particular book requires knowledge of the arrangement of the books.

If we arrange some data in appropriate sequence, then it forms a **Structure** and gives us a meaning. This meaning is called **information**. The basic unit of information in computer Science is a bit, binary digit. So we found two things in information: One is **Data** and the other is **Structure**

Relationship between Data Structures vs Algorithms

Data structures and algorithms are two interrelated concepts in computer science. **Data structures refer to the organization, storage, and retrieval of data, while algorithms refer to the set of instructions used to solve a particular problem or perform a specific task.**

Data structures provide a framework for storing and accessing data that algorithms can operate on.

Data structures and algorithms are often used together to create efficient and effective software solutions. For example, a sorting algorithm can be used to sort data in an array, while a search algorithm can be used to find a particular item in a linked list.

Efficient data structures and algorithms are critical to the performance of software systems. The choice of data structure and algorithm depends on the type of data being processed and the nature of the problem to be solved. By selecting the most appropriate data structure and algorithm, software developers can create high-performance solutions that are both efficient and effective.

Difference and Relationship between Data Structures vs Algorithms

Aspect	Data Structures	Algorithms
Definition	The way data is organized, stored, and retrieved	A set of instructions used to solve a specific problem
Purpose	Provides an efficient way to organize and store data for easy retrieval and modification	Provides a systematic approach to solving problems by breaking them down into smaller, more manageable steps
Operations	Insertion, Deletion, Search, Update, Traverse, etc.	Sorting, Searching, Optimization, Pathfinding, etc.
Importance	Essential for efficient data storage and retrieval	Crucial for developing efficient software solutions
Relationship	Data structures provide a framework for storing and accessing data that algorithms can operate on	Algorithms often operate on data structures to process or manipulate data
Performance	The efficiency of data structures determines the efficiency of algorithms that operate on them	The choice of algorithm can have a significant impact on the performance of the software solution
Examples	Array, Linked List, Stack, Queue, Tree, Graph, Hash Table, etc.	Sorting, Searching, Graph Traversal, Dynamic Programming, Divide and Conquer, etc.

Relationship between Data Structures vs Algorithms Time Complexity

- **Data structures** are the foundation of efficient algorithms and play a crucial role in determining the time complexity of a piece of code.
- Different data structures may have different time and space complexity characteristics for various operations, which can impact the performance of an algorithm. Time complexity refers to the amount of time it takes for an algorithm to complete, and it is a measure of how the runtime of an algorithm grows as the input size increases.
- The choice of data structure is often determined by the specific algorithm being implemented and can significantly impact the time complexity of a solution, making it faster or slower.

Example: Arrays are the simplest and most basic data structure. They are contiguous blocks of memory that store elements in a linear fashion. Accessing, inserting, and deleting elements from an array have a constant time complexity of $O(1)$, meaning the time taken is independent of the size of the array. However, inserting or deleting elements from the middle of an array can be expensive, as it requires shifting all the elements after the insertion or deletion point.

2.2.2 Abstract Data Type

An **abstract data type (ADT)** is a logical description of the data and its operations. An ADT is known as a user's point of view of data. An ADT is concerned about the possible values of data and interfaces exposed by it. An ADT is not concerned about the actual implementation of the data structure.

Abstract Data Type is a mathematical model of a data structure that specifies the type of data stored, the operations supported on them and the types of parameters of the operations with no implementation. Each programming language has its own way to implement abstract data type.

Differently to Data structures, **Data Structures** are a concrete representations of data and implementation. Data structures serve as the basis for abstract data types (ADT). The ADT defines the logical form of the data type. The data structure implements the physical form of the data type

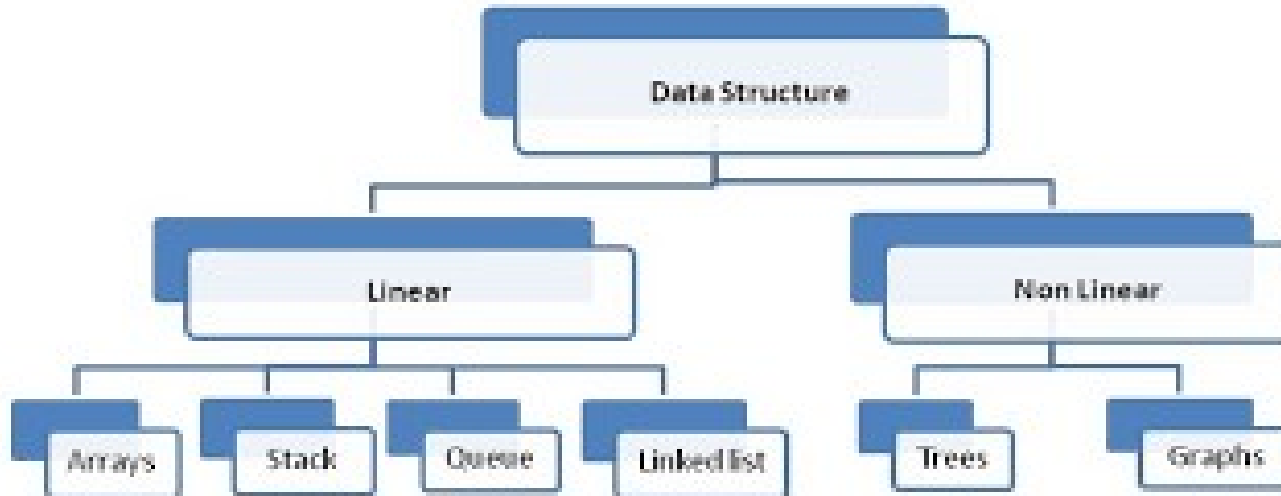
For example:

A user wants to store some integers and find their mean value. ADT for this data structure would have two functions, one for adding integers and another to get the mean value. ADT for this data structure does not talk about how exactly it will be implemented .ADT is more of a logical description,

Each programming language has its own way to implement abstract data type.

2.2.3 Basic types of Data Structures

Anything that can store data can be called as a **data structure**, hence Integer, Float, Boolean, Char etc., all are data structures. They are known as **Primitive Data Structures**. And we also have some complex Data Structures, which are used to store large and connected data and are called **non primitive** data structure or **Abstract Data Structure**. **Example:** Array, Linked List, Tree, Graph, Stack, Queue etc



TYPES OF DATA STRUCTURE

Linear vs No-linear

A linear data structure is a type of Data Structure where the data elements get linearly or sequentially arranged. The elements attach themselves to their next and previous adjacents. The structure involves only a single level- allowing a user to traverse all its components in a single run.

The linear data structure is very easy to understand and implement due to its linear arrangement, for example, stack, array, linked list, queue, etc.

Non-Linear Data Structure is a form of data structure where the data elements don't stay arranged linearly or sequentially. Since the data structure is non-linear, it does not involve a single level. Therefore, a user can't traverse all of its elements in a single run.

The non-linear data structure is not very easy to implement as compared to the linear data structure. The utilization of computer memory is more efficient in this case, for example, graphs and trees.

Linear vs No-linear

Parameter	Linear Data Structure	Non-Linear Data Structure
Arrangement	In a linear data structure, the data elements connect to each other sequentially.	The data elements connect to each other hierarchically.
Complexity of Implementation	The linear data structures are comparatively easier to implement.	The non-linear data structures are comparatively difficult to implement and understand
Levels	A user can find all of the data elements at a single level in a linear data structure.	One can find all the data elements at multiple levels in a non-linear data structure.
Traversal	You can traverse a linear data structure in a single run.	The users need multiple runs to traverse them completely.
Utilization of Memory	It is not very memory-friendly. It means that the linear data structures can't utilize memory very efficiently.	The data structure is memory-friendly. It means that it uses memory very efficiently.
Complexity of Time	The time complexity of this data structure is directly proportional to input size.	Time complexity often remains the same with an increase in its input size.
Applications	Linear data structures work well mainly in the development of application software.	Non-linear data structures work mainly well in image processing and Artificial Intelligence.
Examples	List, Array, Stack, Queue.	Map, Graph, Tree.

2.2.4 Importance of data structures

The data structures have the following importance for programming:

- Data structures study how data are stored in a computer so that operations can be implemented efficiently
- Data structures are especially important when there is a large amount of information to deal with.
- Data structures are conceptual and concrete ways to organize data for efficient storage and manipulation

2.2.5 Data Structure Operations

Beside the simple operations performed on data (unary operators, binary operators and precedence operators) seen previously, the data appearing in our data structures are processed by means of other operations.

The following are some of the most frequently used operations.

Traversing: Accessing each record exactly once so that certain items in the record may be processed.

- **Searching:** Finding the location of the record with a given key value or finding the locations of all records which satisfy one or more conditions
- **Inserting:** Adding a new record to the structure.
- **Deleting:** Removing a record from the structure.
- **Sorting:** Arranging the records in some logical order (e.g., in numerical order according to some NUMBER key, such as social security number or accountnumber).
- **Merging:** Combining the records of two different sorted lists into a single sorted list.

Array

Arrays are the simplest data structures that store items of the same data type.

Array ADT Operations

Below is the API of an array:

1. Adds an element at the **kth** position. Value can be stored in an array at the **kth** position in **O(1)** constant time. We just need to store value at **arr[k]**.
2. Reading the value stored at the **kth** position. Accessing the value stored at some index in the array is also **O(1)** constant time. We just need to read the value stored at **arr[k]**.
3. Substitution of value stored in **kth** position with a new value.
4. **Time complexity: O(1)** constant time.

Output:

0 1 2 3 4 5 6 7 8 9

Applications of Arrays are:

1. Storing data in tabular format.
2. Used in the creation of Matrices.
3. Used in the creation of various higher level data structures like Stacks, Queues, Heaps, HashTables etc.

Note: Problems based on array data structure are discussed later, especially in the Sorting and Searching topics

Example:

```
int main() {  
    }  
    int arr[10];  
    int size= sizeof(arr) / sizeof(int);  
    for (int i = 0; i < size; i++) {  
        arr[i] = i;  
    }  
    for (int i = 0; i < size; i++) {  
        printf("%d ", arr[i]);  
    }  
    return 0;  
}
```

Other Abstract Data Structures

In coming learning units, apart from Array, will be studying various data structures and their API so that the user can use them and their internal implementation.

These include Linked List, Stacks, Queues, Heaps, Hash Tables, Tree, Graph, Hash Table among the others.